

SAGEVM

Assembly Guide

Version	v0.9.8
Organization	Night-Traders-Dev
Architectures	SVM (Stack) & SRVM (Register/RISC-V)
Document Type	Technical Guide & Reference
Status	Draft / Active Development

The Definitive Guide to writing assembly for the Sage General Virtual Machine (SGVM). This document explores the low-level mechanics of the dual-VM architecture, providing comprehensive tutorials and examples for both the stack-based SVM and the RISC-V mapped SRVM. From basic arithmetic to advanced system-level bootloaders and GPU command streams.

SageVM implements a unique dual-VM architecture, allowing high-level Sage code to be compiled into two distinct execution substrates:

- **SVM (Stack Virtual Machine):** Optimized for binary density and portable execution. Uses an implicit operand stack.
- **SRVM (Register Virtual Machine):** Optimized for speed and JIT friendliness. Maps directly to the RV64I (RISC-V) specification.

2.1 The SVM Stack Logic

In SVM, all operations happen at the top of an implicit LIFO stack.

- **Pushing:** `CONSTANT 0` pushes the first item from the constant pool.
- **Operating:** `ADD` pops two values and pushes their sum.
- **Scoping:** Managed via `PUSH_ENV` and `POP_ENV`.

2.2 The SRVM Register Logic

SRVM uses 32 general-purpose 64-bit registers (x0-x31).

- **x0 (zero):** Hardwired to 0.
- **x10-x17 (a0-a7):** Function arguments and return values.
- **VMSYS:** A special `SYSTEM` instruction (0x73) for calling VM Services (Print, Halt, etc.).

3. Real-World Examples: The Basics

3.1 Basic Math and Output

Objective: Compute $(10 + 20) * 2$ and print the result.

Pure Sage Equivalent

```
print (10 + 20) * 2
```

SVM Assembly

```
CONSTANT 10
CONSTANT 20
ADD
CONSTANT 2
MUL
PRINT
```

SRVM (RISC-V) Assembly

```
ldc x10, 10
ldc x11, 20
add x10, x10, x11
ldc x11, 2
mul x10, x10, x11
vmsys x10, 0x09 # VM_PRINT
```

3.2 Control Flow (While Loop)

Objective: Print numbers from 1 to 5.

Pure Sage

```
var i = 1
while i <= 5:
    print i
    i = i + 1
```

SVM Assembly

```
CONSTANT 1
DEFINE_GLOBAL "i"
LOOP_START:
  GET_GLOBAL "i"
  CONSTANT 5
  LESS_EQUAL
  JUMP_IF_FALSE EXIT
  GET_GLOBAL "i"
  PRINT
  GET_GLOBAL "i"
  CONSTANT 1
  ADD
  SET_GLOBAL "i"
  JUMP LOOP_START
EXIT:
HALT
```

4. Advanced Examples: Objects and Exceptions

4.1 Object Orientation

SageVM handles classes via method tables and property maps.

```
class Point:
    proc init(self, x, y):
        self.x = x
        self.y = y
```

- **SVM Implementation:** Uses CLASS, METHOD, and SET_PROPERTY opcodes.
- **SRVM Implementation:** Uses vmsys with OBJ_NEW_CLASS and OBJ_METHOD_BIND markers.

4.2 Exception Handling

Exceptions use a dedicated handler stack for unwinding.

SVM Pattern

```
SETUP_TRY CATCH_LABEL
CONSTANT "Error!"
RAISE
END_TRY
JUMP EXIT
CATCH_LABEL:
    PRINT
EXIT:
```

5. Advanced System & GPU Interfacing

5.1 Advanced SRVM Bootloader

Demonstrates low-level machine state initialization in RISC-V.

```
ldc x10, 0x01      # Const Index 1: Memory Block
mv x2, x10         # Set Stack Pointer (sp)
ldc x3, 0x02       # Const Index 2: Global Env
vmsys x0, 0x02     # VMO_PUSH_ENV
ldc x10, 0x03      # Index 3: kernel_main
vmsys x10, 0x04    # VMO_CALL
vmsys x0, 0x01     # VMO_HALT
```

5.2 Advanced SVM GPU Bridge (Vulkan)

Managing a Vulkan render pass via native VM opcodes.

```
OP_GPU_POLL_EVENTS
CONSTANT 0; OP_GPU_ACQUIRE_IMG
CONSTANT 1; OP_GPU_BEGIN_COMMANDS
# ... Begin Render Pass ...
OP_GPU_CMD_BEGIN_RP
CONSTANT 6; OP_GPU_CMD_BIND_GP
# ... Draw Call ...
OP_GPU_CMD_DRAW
OP_GPU_CMD_END_RP
OP_GPU_END_COMMANDS
OP_GPU_PRESENT
```

6. Technical Reference & Pro Tips

6.1 Constant Pool Encoding

Type	Byte	Description
Double	0x01	IEEE 754 double-precision (8 bytes)
String	0x02	Length-prefixed (2-byte BE) UTF-8 data
Int32	0x03	32-bit big-endian integer

6.2 Pro Tips

- 1. Halt is Mandatory:** Always end your main chunk with HALT to prevent executing uninitialized memory.
- 2. Native Bridge:** Use OP_CALL on global objects like math or io to switch to high-performance C implementations.
- 3. Debugging:** Leverage `sagevm hex binary.sgm` to inspect the exact byte layout of your binaries.

End of Document · SageVM Assembly Guide v0.9.8 · Night-Traders-Dev